

Task 1: Create a Table→

A) Create a database named Company:

```
CREATE DATABASE Company;

USE Company;
```

B) Create a table named Employees :

```
CREATE DATABASE Company;

USE Company;

CREATE TABLE Employees (
    EmployeeID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Department VARCHAR(50),
    Salary DECIMAL(10, 2),
    HireDate DATE
);
```

Query OK, 1 row affected (0.01 sec) -- For: CREATE DATABASE Company;
Database changed -- For: USE Company;
Query OK, 0 rows affected (0.03 sec) -- For: CREATE TABLE Employees (...);

Task 2: DDL Queries on the Employees Table→

A) Describe the structure of the table:

```
CREATE DATABASE Company;

USE Company;

CREATE TABLE Employees (
    EmployeeID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Department VARCHAR(50),
    Salary DECIMAL(10, 2),
    HireDate DATE
);

DESCRIBE Employees;
```

Field	Type	Null	Key	Default	Extra
EmployeeID	int	NO	PRI	NULL	auto_increment
FirstName	varchar(50)	NO		NULL	
LastName	varchar(50)	NO		NULL	
Email	varchar(100)	YES	UNI	NULL	
Department	varchar(50)	YES		NULL	
Salary	decimal(10,2)	YES		NULL	
HireDate	date	YES		NULL	

B) Rename the table to Staff:

```

CREATE DATABASE Company;

USE Company;

CREATE TABLE Employees (
  EmployeeID INT AUTO_INCREMENT PRIMARY KEY,
  FirstName VARCHAR(50) NOT NULL,
  LastName VARCHAR(50) NOT NULL,
  Email VARCHAR(100) UNIQUE,
  Department VARCHAR(50),
  Salary DECIMAL(10, 2),
  HireDate DATE
);

DESCRIBE Employees;

RENAME TABLE Employees TO Staff;

```

C) Add a new column PhoneNumber (VARCHAR(15)):

```

CREATE DATABASE Company;

USE Company;

CREATE TABLE Employees (
  EmployeeID INT AUTO_INCREMENT PRIMARY KEY,
  FirstName VARCHAR(50) NOT NULL,
  LastName VARCHAR(50) NOT NULL,
  Email VARCHAR(100) UNIQUE,
  Department VARCHAR(50),
  Salary DECIMAL(10, 2),
  HireDate DATE
);

DESCRIBE Employees;

RENAME TABLE Employees TO Staff;

ALTER TABLE Staff ADD PhoneNumber VARCHAR(15);

```

Field	Type	Null	Key	Default	Extra
EmployeeID	int	NO	PRI	NULL	auto_increment
FirstName	varchar(50)	NO		NULL	
LastName	varchar(50)	NO		NULL	
Email	varchar(100)	YES	UNI	NULL	
Department	varchar(50)	YES		NULL	
Salary	decimal(10,2)	YES		NULL	
HireDate	date	YES		NULL	
PhoneNumber	varchar(15)	YES		NULL	

8 rows in set (0.00 sec)

D) Modify the data type of Salary to DECIMAL(12,2):

```

CREATE DATABASE Company;

USE Company;

CREATE TABLE Employees (
  EmployeeID INT AUTO_INCREMENT PRIMARY KEY,
  FirstName VARCHAR(50) NOT NULL,
  LastName VARCHAR(50) NOT NULL,
  Email VARCHAR(100) UNIQUE,
  Department VARCHAR(50),
  Salary DECIMAL(10, 2),
  HireDate DATE
);

DESCRIBE Employees;

RENAME TABLE Employees TO Staff;

ALTER TABLE Staff ADD PhoneNumber VARCHAR(15);

ALTER TABLE Staff MODIFY Salary DECIMAL(12, 2);

```

Field	Type	Null	Key	Default	Extra
EmployeeID	int	NO	PRI		auto_increment
FirstName	varchar(50)	NO			
LastName	varchar(50)	NO			
Email	varchar(100)	YES	UNI		
Department	varchar(50)	YES			
Salary	decimal(12,2)	YES			
HireDate	date	YES			
PhoneNumber	varchar(15)	YES			

8 rows in set (0.01 sec)

E)Add a DEFAULT value of 'General' to the Department column:

```

ALTER TABLE Staff MODIFY Salary DECIMAL(12, 2);

ALTER TABLE Staff ALTER Department SET DEFAULT 'General';

```

ID	NAME	DEPARTMENT	SALARY	EMAIL	PHONENO
1	YASHIKA	CSE	45000.00	anshika123@gmail.com	NULL
2	GAURI	NULL	45000.00	gauri163@gmail.com	NULL
3	priya	NULL	65000.00	priya163@gmail.com	NULL
4	SUMAN	NULL	10000.00	suman173@gmail.com	NULL

F)Drop the Phone Number column:

```

INSERT INTO Staff (ID, NAME, SALARY, EMAIL)
VALUES
(2, 'GAURI', 45000.00, 'gauri163@gmail.com'),
(3, 'priya', 65000.00, 'priya163@gmail.com'),
(4, 'SUMAN', 10000.00, 'suman173@gmail.com');

UPDATE Staff SET NAME = 'YASHIKA' WHERE ID = 1;

ALTER TABLE Staff DROP COLUMN PHONENO;

SELECT * FROM Staff;

```

ID	NAME	DEPARTMENT	SALARY	EMAIL
1	YASHIKA	CSE	45000.00	anshika123@gmail.com
2	GAURI	NULL	45000.00	gauri163@gmail.com
3	priya	NULL	65000.00	priya163@gmail.com
4	SUMAN	NULL	10000.00	suman173@gmail.com

4 rows in set (0.00 sec)

G) Add a CHECK constraint to ensure Salary is greater than 20000:

```
ALTER TABLE Staff ADD CONSTRAINT chk_salary CHECK (SALARY > 20000);
```

```
INSERT INTO Staff (ID, NAME, DEPARTMENT, SALARY, EMAIL)  
VALUES (1, 'YASHIKA', 'CSE', 45000.00, 'yashika123@gmail.com');
```

```
INSERT INTO Staff (ID, NAME, SALARY, EMAIL)
```

```
VALUES
```

```
(2, 'GAURI', 45000.00, 'gauri163@gmail.com'),  
(3, 'priya', 65000.00, 'priya163@gmail.com'),  
(4, 'SUMAN', 25000.00, 'suman173@gmail.com');
```

```
SELECT * FROM Staff;
```

ID	NAME	DEPARTMENT	SALARY	EMAIL
1	YASHIKA	CSE	45000.00	yashika123@gmail.com
2	GAURI	NULL	45000.00	gauri163@gmail.com
3	priya	NULL	65000.00	priya163@gmail.com
4	SUMAN	NULL	25000.00	suman173@gmail.com

H) Rename the column FirstName to GivenName:

```
ALTER TABLE Staff ADD CONSTRAINT chk_salary CHECK (SALARY > 20000);
```

```
ALTER TABLE Staff RENAME COLUMN NAME TO GivenName;
```

```
INSERT INTO Staff (ID, GivenName, DEPARTMENT, SALARY, EMAIL)  
VALUES (1, 'YASHIKA', 'CSE', 45000.00, 'yashika123@gmail.com');
```

```
INSERT INTO Staff (ID, GivenName, SALARY, EMAIL)
```

```
VALUES
```

```
(2, 'GAURI', 45000.00, 'gauri163@gmail.com'),  
(3, 'priya', 65000.00, 'priya163@gmail.com'),  
(4, 'SUMAN', 25000.00, 'suman173@gmail.com');
```

```
SELECT * FROM Staff;
```

ID	GivenName	DEPARTMENT	SALARY	EMAIL
1	YASHIKA	CSE	45000.00	yashika123@gmail.com
2	GAURI	NULL	45000.00	gauri163@gmail.com
3	priya	NULL	65000.00	priya163@gmail.com
4	SUMAN	NULL	25000.00	suman173@gmail.com

4 rows in set (0.00 sec)

I) Drop the chk_Salary constraint:

```
ALTER TABLE Staff DROP CONSTRAINT chk_salary;
```

```
ALTER TABLE Staff RENAME COLUMN NAME TO GivenName;
```

```
CREATE INDEX idx_email ON Staff(EMAIL);
```

```
INSERT INTO Staff (ID, GivenName, DEPARTMENT, SALARY, EMAIL)  
VALUES (1, 'YASHIKA', 'CSE', 45000.00, 'yashika123@gmail.com');
```

```
INSERT INTO Staff (ID, GivenName, SALARY, EMAIL)
```

```
VALUES
```

```
(2, 'GAURI', 45000.00, 'gauri163@gmail.com'),  
(3, 'priya', 65000.00, 'priya163@gmail.com'),  
(4, 'SUMAN', 10000.00, 'suman173@gmail.com');
```

```
SELECT * FROM Staff;
```

ID	GivenName	DEPARTMENT	SALARY	EMAIL
1	YASHIKA	CSE	45000.00	yashika123@gmail.com
2	GAURI	NULL	45000.00	gauri163@gmail.com
3	priya	NULL	65000.00	priya163@gmail.com
4	SUMAN	NULL	10000.00	suman173@gmail.com

4 rows in set (0.00 sec)

J) Create an index on the Email column:

ID	GivenName	DEPARTMENT	SALARY	EMAIL
1	YASHIKA	CSE	45000.00	yashika123@gmail.com
2	GAURI	NULL	45000.00	gauri163@gmail.com
3	priya	NULL	65000.00	priya163@gmail.com
4	SUMAN	NULL	25000.00	suman173@gmail.com

4 rows in set (0.00 sec)

K) Drop the index on the Email column:

```
CREATE INDEX idx_email ON Staff(EMAIL);
```

```
ALTER TABLE Staff DROP INDEX idx_email;
```

```
INSERT INTO Staff (ID, GivenName, DEPARTMENT, SALARY, EMAIL)  
VALUES (1, 'YASHIKA', 'CSE', 45000.00, 'yashika123@gmail.com');
```

```
INSERT INTO Staff (ID, GivenName, SALARY, EMAIL)  
VALUES  
(2, 'GAURI', 45000.00, 'gauri163@gmail.com'),  
(3, 'priya', 65000.00, 'priya163@gmail.com'),  
(4, 'SUMAN', 10000.00, 'suman173@gmail.com');
```

```
SELECT * FROM Staff;
```

ID	GivenName	DEPARTMENT	SALARY	EMAIL
1	YASHIKA	CSE	45000.00	yashika123@gmail.com
2	GAURI	NULL	45000.00	gauri163@gmail.com
3	priya	NULL	65000.00	priya163@gmail.com
4	SUMAN	NULL	10000.00	suman173@gmail.com

4 rows in set (0.00 sec)

L) Truncate the table to remove all rows without affecting the structure:

```
CREATE INDEX idx_email ON Staff(EMAIL);  
  
ALTER TABLE Staff DROP INDEX idx_email;
```

```
INSERT INTO Staff (ID, GivenName, DEPARTMENT, SALARY, EMAIL)  
VALUES (1, 'YASHIKA', 'CSE', 45000.00, 'yashika123@gmail.com');
```

```
INSERT INTO Staff (ID, GivenName, SALARY, EMAIL)  
VALUES  
(2, 'GAURI', 45000.00, 'gauri163@gmail.com'),  
(3, 'priya', 65000.00, 'priya163@gmail.com'),  
(4, 'SUMAN', 10000.00, 'suman173@gmail.com');
```

```
TRUNCATE TABLE Staff;
```

```
Query OK, 0 rows affected (0.02 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
Query OK, 0 rows affected (0.01 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
Query OK, 0 rows affected (0.02 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
Query OK, 0 rows affected (0.01 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
Query OK, 1 row affected (0.01 sec)
```

```
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

```
Query OK, 0 rows affected (0.04 sec)
```

M) Drop the Staff table completely:

```
ALTER TABLE Staff DROP INDEX idx_email;
```

```
INSERT INTO Staff (ID, GivenName, DEPARTMENT, SALARY, EMAIL)  
VALUES (1, 'YASHIKA', 'CSE', 45000.00, 'yashika123@gmail.com');
```

```
INSERT INTO Staff (ID, GivenName, SALARY, EMAIL)  
VALUES  
(2, 'GAURI', 45000.00, 'gauri163@gmail.com'),  
(3, 'priya', 65000.00, 'priya163@gmail.com'),  
(4, 'SUMAN', 10000.00, 'suman173@gmail.com');
```

```
TRUNCATE TABLE Staff;
```

```
DROP TABLE Staff;
```

```
Query OK, 0 rows affected (0.02 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
Query OK, 0 rows affected (0.01 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
Query OK, 0 rows affected (0.01 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
Query OK, 1 row affected (0.01 sec)
```

```
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

```
Query OK, 0 rows affected (0.04 sec)
```

```
Query OK, 0 rows affected (0.02 sec)
```

3. Conceptual Questions:

1. What is the difference between TRUNCATE and DELETE?

ANS-

DELETE

TRUNCATE

Deletes selected rows or all rows from a table.

Deletes **all rows** from a table.

Can use WHERE condition.

Cannot use WHERE.

ANS-

DELETE

Example: DELETE
FROM Student
WHERE id=5;

Slower for deleting all records because rows are deleted individually.

Can be rolled back in many DBMSs when used within a transaction.

Table structure remains.

DELETE is a **DML** command.

TRUNCATE

Example:
TRUNCATE
TABLE Student;

Faster because it removes all records at once.

Rollback behavior depends on the DBMS.

Table structure remains.

TRUNCATE is generally classified as **DDL**.

2. Explain the use of the CHECK constraint.

ANS- The CHECK constraint is used to ensure that the values entered into a table satisfy a specific condition.

It prevents invalid data from being inserted into the table.

Syntax:

```
CREATE TABLE Student (  
    id INT,  
    age INT CHECK (age >= 18)  
);
```

Here, CHECK (age >= 18) means that the student's age must be **18 or above**.

3. Explain the use of the CHECK constraint.

ANS- The CHECK constraint is used to ensure that the values entered into a table satisfy a specific condition.

It prevents invalid data from being inserted into the table.

Syntax:

```
CREATE TABLE Student (  
    id INT,  
    age INT CHECK (age >= 18)  
);
```

Here, CHECK (age >= 18) means that the student's age must be **18 or above**.

4. Exercises:

a) Perform at least 5 DDL operations on the `Projects` table

```
ALTER TABLE Projects ADD Description VARCHAR(255);
```

```
ALTER TABLE Projects MODIFY ProjectName VARCHAR(150) NOT NULL;
```

```
ALTER TABLE Projects RENAME COLUMN Description TO ProjectDescription;
```

```
ALTER TABLE Projects DROP COLUMN ProjectDescription;
```

```
ALTER TABLE Projects ADD Status VARCHAR(20);
```

b) Retrieve the names of all columns in the `Staff` table

```
SELECT COLUMN_NAME  
FROM INFORMATION_SCHEMA.COLUMNS  
WHERE TABLE_SCHEMA = DATABASE()  
AND TABLE_NAME = 'Staff';
```

c) Modify the `Department` column to allow NULL values

```
ALTER TABLE Staff  
MODIFY Department VARCHAR(100) NULL;
```